

Akshath Reddy Yennam

github.com/Akshath47 | linkedin.com/in/akshathyennam | akshathyennam@gmail.com | akshathyennam.com

EDUCATION

- 2024 – 2028 MEng Computing at **Imperial College London** (First Class in Year 1 & Year 2)
Relevant Coursework: Linear Algebra, Probability & Statistics, Algorithms & Data Structures, Operating Systems, Machine Learning, Software Engineering Design, Calculus, Networks & Communications
- 2022 – 2024 A Levels at Sancta Maria International School (Valedictorian, 4 A*, 1 A)

WORK EXPERIENCE

Man Group | *Software Engineering Intern* Jun 2026 – Sep 2026 (current) | *London, UK*

- Built a reusable data lineage integration layer connecting Investment Analytics ETL pipelines to the firm's DataHub platform, with emission handling per source and sink type, removing the need for bespoke integration across a roughly 100-pipeline estate.
- Implement declarative data quality checks (freshness, schema, null/completeness) running at every ETL load step, triggered by Kafka events, catching data issues before client delivery.
- Designed lineage emission to fail gracefully so instrumentation faults never block a production ETL run, with checks executed in parallel across Ray clusters in an isolated Kubernetes namespace to meet client data separation requirements.
- Embedded both layers into a new config-driven pipeline class so future pipelines inherit lineage and quality checks by default, with no per-pipeline setup.

Pulpoo | *Backend Engineer, Part-time* Jan 2026 – Jun 2026 | *Remote*

- Extracted a reusable omnichannel support platform core from a production system, with an overlay layer letting client-specific workflows build against one core interface without forking it, supporting expansion to new clients and channels.
- Built an evaluation and observability layer (prompt versioning, regression testing, per-agent cost tracking) for a live Samsung Mexico deployment, gating model downgrades on test results to cut inference cost by 30% with no regression on the eval suite.
- Cut live response latency 70% by moving memory extraction and knowledge graph (Neo4j) writes off the synchronous path into an async job queue, isolating slow persistence work from user-facing replies.

ValueLabs | *AI Engineering Intern* Jul 2025 – Sep 2025 | *Hyderabad, India*

- Rebuilt a JD-to-resume semantic search system on the Qdrant vector database, cutting query response times from about 90s to about 6s through caching and search parameter tuning.
- Built a hybrid retrieval pipeline (vector plus keyword search, multi-query rewriting, reciprocal rank fusion) for a sales meeting assistant, validated against vector-only retrieval on a held-out query set.

PROJECTS

WACC Compiler | *Scala, ARM, DevOps* Jan 2026 – Mar 2026

- Built a complete compiler in Scala for WACC (a C-like imperative language), covering lexing, parsing, semantic analysis, IR generation, and ARM code generation.
- Extended beyond spec with a middle-end optimisation suite: CFG construction with dead-block elimination and jump threading, and dataflow-based constant propagation and dead code elimination, cutting generated assembly size by up to 35% on redundancy-heavy benchmarks.
- Implemented a liveness-directed graph-colouring register allocator with four spill heuristics, plus copy propagation with allocator coalescing, reducing runtime by 25% on a spill-heavy benchmark.
- Ran a controlled study across 6 benchmarks and 5 optimisation configs in a Dockerised harness, measuring runtime, assembly size, and spill counts: constant propagation unlocked further dead code elimination and cut spills.

PintOS – Operating System | *C, Concurrency* Oct 2025 – Dec 2025

- Implemented core kernel subsystems in C: a priority-based thread scheduler extended with nested priority donation, alongside synchronisation primitives for safe concurrency.
- Handled user process lifecycle management through syscall implementations for safe process execution, validating user-space memory accesses to maintain kernel/user isolation.
- Extended virtual memory with demand paging, lazy loading of executable segments, and a swap-backed enhanced clock policy over reference and dirty bits, cutting initial process memory usage by over 95% on the given benchmark tests.

SKILLS

Languages	Python, C, C++, Scala, Haskell, Java, Kotlin, TypeScript/JavaScript, SQL
Systems & Infrastructure	Kubernetes, Kafka, Ray, Docker, Linux, CI/CD, Git
Data & Frameworks	PostgreSQL, Neo4j, Redis, Qdrant, PyTorch, LangGraph, React, Node.js